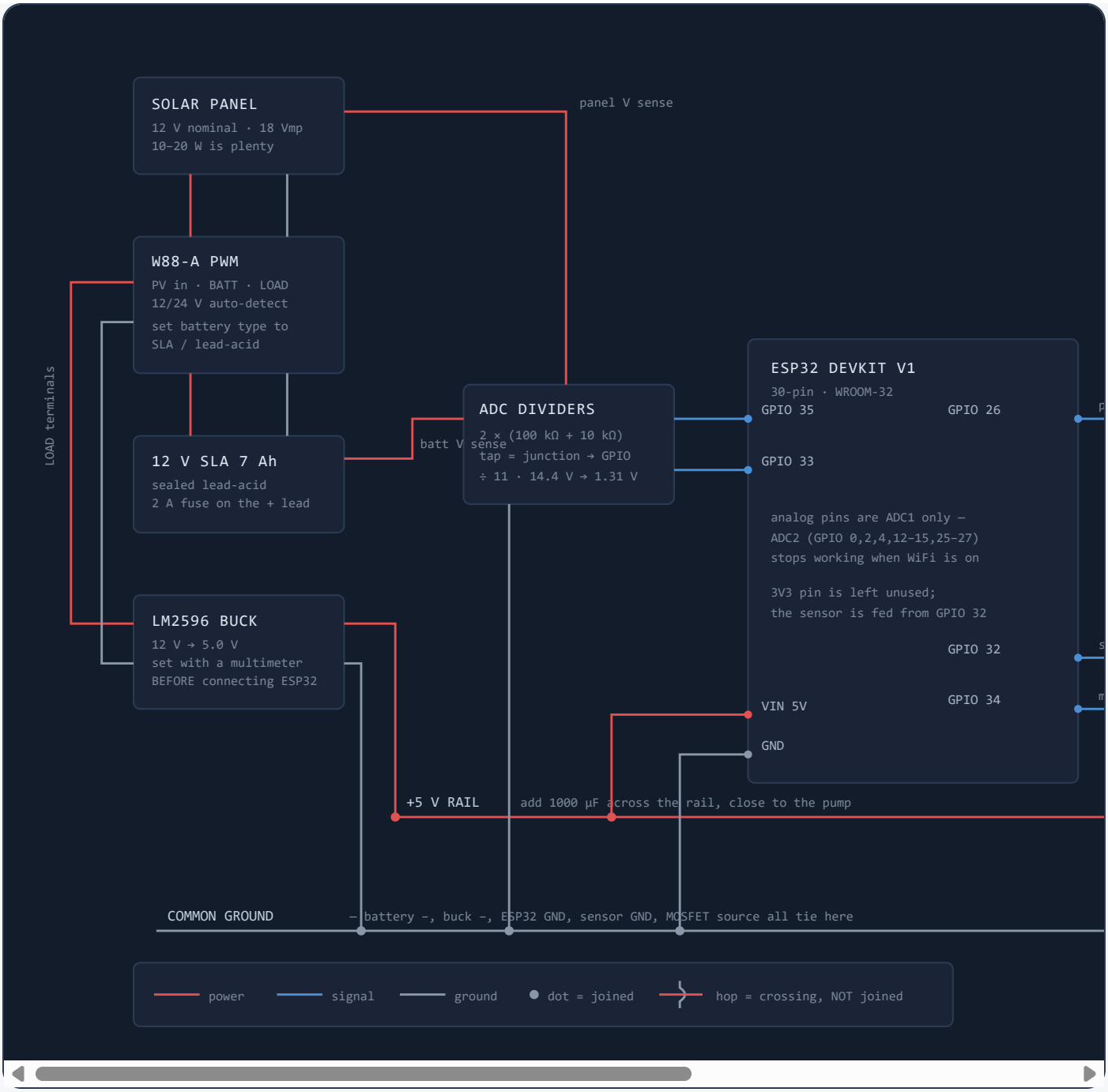
WBACFSPWI — Hardware Build Sheet

Wiring, pin map and firmware for the ESP32 node that feeds `public/api/device/report.php` and reads `pull-schedule.php`.

> **Short answer: yes, but not with the parts exactly as listed.**
>
> Two blockers. (1) The **W88-A is a 12 V/24 V lead-acid PWM controller** — a 4×AA holder is 6 V, and alkaline AAs must never be charged. It needs a 12 V SLA battery. (2) There is **no switching device in your list** — an ESP32 GPIO sources ~20 mA, your pump wants 150 mA+. Wiring the pump straight to a pin kills the pin. Add a logic-level MOSFET. Everything else you own is usable.

Bonus: with a 12 V battery, `config/device.php` already matches — `BATTERY_MIN_VOLTS 11.0`, `BATTERY_MAX_VOLTS 14.4`, `ALERT_LOW_BATTERY_VOLTS 11.5` are lead-acid numbers. Zero PHP changes needed.

Wiring diagram



Parts

PART	STATUS	NOTES
Solar panel	HAVE	Must be a 12 V-nominal panel (~18 V open circuit) for the W88-A. A 6 V panel will not charge a 12 V battery.
W88-A charge controller	HAVE	Set battery type to SLA/lead-acid. Connect in order: **battery first, then panel, then load** — controllers detect system voltage from the battery.

PART	STATUS	NOTES
ESP32	HAVE	Powered from the 5 V rail into VIN (the onboard AMS1117 makes 3.3 V).
HW-080 moisture sensor	HAVE	Resistive — it corrodes. Fine for the defense demo; swap to a capacitive v1.2 if this runs for months.
Micro water pump 3–6 V	HAVE	Runs directly off the 5 V rail, switched low-side by the MOSFET.
Breadboard + jumpers	HAVE	Signals only. Keep the pump current off the breadboard rails — see the warning below.
12 V 7 Ah SLA battery	BUY	The single required purchase. This is what makes the W88-A and the dashboard's voltage thresholds work.
LM2596 buck module	BUY	12 V → 5 V. Set the output to 5.0 V with a multimeter *before* the ESP32 is attached.
IRLZ44N MOSFET	BUY	Logic-level. A 5 V relay module also works if you already own one.
1N5819 diode, 1000 µF cap	BUY	Flyback across the pump; bulk cap across the 5 V rail. Both prevent random ESP32 resets.
2× 100 kΩ, 2× 10 kΩ, 220 Ω, 10 kΩ	BUY	¼ W. Two dividers, one gate resistor, one gate pulldown.
2 A inline fuse	BUY	On the battery + lead. An SLA can push 100+ A into a short.
4-slot AA holder	NOT IN CIRCUIT	6 V is incompatible with the W88-A, and charging alkalines is a fire risk. Keep it for bench-testing the pump alone.

Pin map

ESP32 PIN	GOES TO	WHY THIS PIN
`VIN`	+5 V rail	Board regulator handles 5 V in. Do not also feed 3V3 from anything.
`GND`	Common ground	The wire everyone forgets. Sensor readings float without it.

ESP32 PIN	GOES TO	WHY THIS PIN
`GPIO 34`	HW-080 `AO`	ADC1, input-only. Analog moisture.
`GPIO 32`	HW-080 `VCC`	Output. Powers the probe only during a reading, which massively slows corrosion.
`GPIO 35`	Battery divider tap	ADC1. 12 V ÷ 11 → ~1.1–1.3 V, safely inside the ADC's linear range.
`GPIO 33`	Panel divider tap	ADC1. 21 V open-circuit ÷ 11 → 1.9 V, still safe.
`GPIO 26`	MOSFET gate (via 220 Ω)	Output, no boot-strapping role. 10 kΩ pulldown keeps the pump off during reset.

The ADC2 trap. GPIO 0, 2, 4, 12–15 and 25–27 are ADC2 — `analogRead()` on them returns garbage or hangs while WiFi is active. Every analog pin above is ADC1 on purpose. If your readings suddenly go to zero after adding WiFi, this is why.

Divider math

```
// 100 kΩ from battery+ to the tap, 10 kΩ from the tap to GND
tap = V_batt × 10k / (100k + 10k) = V_batt / 11

  14.4 V (absorption)  → 1.31 V   safe
  12.6 V (full rest)   → 1.15 V
  11.5 V (alert level) → 1.05 V
  11.0 V (0 % on dash) → 1.00 V

// in firmware, read millivolts (already factory-calibrated) then scale
float v = analogReadMilliVolts(35) / 1000.0f * 11.0f;
```

Trim the 11.0 constant once against a multimeter reading — resistor tolerance is ±5 %.

ESP32 firmware

Matches the exact field names in `report.php` and the response shape of `pull-schedule.php`.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>   // Library Manager: "ArduinoJson" by Benoit Blanchon
#include <time.h>

const char* SSID    = "YOUR_WIFI";
```

```
const char* PASS    = "YOUR_PASS";
const char* HOST    = "http://192.168.1.10/WBACFSPWI.../public";  // XAMPP PC's LAN IP
const char* API_KEY = "dev-local-device-key";                     // config/device.php

// --- pins ---
const int PIN_PUMP = 26, PIN_SOIL = 34, PIN_SOIL_PWR = 32;
const int PIN_VBAT = 35, PIN_VSOL = 33;
const float DIVIDER = 11.0;

// --- calibrate these with a dry probe and a probe in water ---
const int SOIL_DRY = 3100, SOIL_WET = 1300;

struct Sched { int id, startMin, durMin; uint8_t days; };
Sched  scheds[10]; int nSched = 0;
bool   pumpOn = false; int activeSchedId = 0;
unsigned long tReport = 0, tPull = 0;

int dayBit(int tm_wday) {          // tm_wday: 0=Sun. DB order: mon..sun = bit0..bit6
  const int map[] = {6,0,1,2,3,4,5};
  return 1 << map[tm_wday];
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_PUMP, OUTPUT);     digitalWrite(PIN_PUMP, LOW);
  pinMode(PIN_SOIL_PWR, OUTPUT); digitalWrite(PIN_SOIL_PWR, LOW);
  analogSetPinAttenuation(PIN_SOIL, ADC_11db);
  analogSetPinAttenuation(PIN_VBAT, ADC_11db);
  analogSetPinAttenuation(PIN_VSOL, ADC_11db);

  WiFi.begin(SSID, PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(400); Serial.print("."); }
  Serial.println(WiFi.localIP());

  configTime(8*3600, 0, "pool.ntp.org");   // UTC+8, Philippines
  struct tm t; while (!getLocalTime(&t)) delay(500);
  pullSchedule();
}

float readMoisture() {
  digitalWrite(PIN_SOIL_PWR, HIGH); delay(500);   // let the probe settle
  long sum = 0; for (int i=0;i<16;i++){ sum += analogRead(PIN_SOIL); delay(10); }
  digitalWrite(PIN_SOIL_PWR, LOW);
  int raw = sum / 16;                              // dry = high, wet = low
  float pct = 100.0 * (SOIL_DRY - raw) / (float)(SOIL_DRY - SOIL_WET);
  return constrain(pct, 0, 100);
}
```

```
float readVolts(int pin) {
  long mv = 0; for (int i=0;i<16;i++) mv += analogReadMilliVolts(pin);
  return (mv / 16.0) / 1000.0 * DIVIDER;
}

void pullSchedule() {
  HTTPClient http;
  http.begin(String(HOST) + "/api/device/pull-schedule.php");
  http.addHeader("X-API-Key", API_KEY);
  if (http.GET() != 200) { http.end(); return; }

  StaticJsonDocument<2048> doc;
  if (deserializeJson(doc, http.getString())) { http.end(); return; }
  http.end();

  nSched = 0;
  const char* names[] = {"mon","tue","wed","thu","fri","sat","sun"};
  for (JsonObject s : doc["schedules"].as<JsonArray>()) {
    if (nSched >= 10) break;
    const char* st = s["start_time"];              // "HH:MM"
    Sched &x = scheds[nSched];
    x.id       = s["id"];
    x.startMin = atoi(st) * 60 + atoi(st + 3);
    x.durMin   = s["duration_minutes"];
    x.days     = 0;
    for (JsonVariant d : s["days_of_week"].as<JsonArray>())
      for (int i=0;i<7;i++) if (!strcmp(d, names[i])) x.days |= (1<<i);
    nSched++;
  }
  Serial.printf("schedules: %d\n", nSched);
}

void sendReport(float soil, float vbat, float vsol) {
  HTTPClient http;
  http.begin(String(HOST) + "/api/device/report.php");
  http.addHeader("Content-Type", "application/json");
  http.addHeader("X-API-Key", API_KEY);

  StaticJsonDocument<256> d;
  d["soil_moisture"]   = round(soil*10)/10.0;
  d["battery_voltage"] = round(vbat*100)/100.0;
  d["solar_output"]    = round(vsol*10)/10.0;
  d["pump_state"]      = pumpOn ? "on" : "off";
  if (pumpOn && activeSchedId) d["schedule_id"] = activeSchedId;

  String body; serializeJson(d, body);
  Serial.printf("POST %d %s\n", http.POST(body), body.c_str());
  http.end();
```

```
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) { WiFi.reconnect(); delay(2000); return; }
  unsigned long now = millis();

  if (now - tPull > 120000) { tPull = now; pullSchedule(); }   // resync every 2 min

  struct tm t; getLocalTime(&t);
  int mins = t.tm_hour*60 + t.tm_min;

  bool shouldRun = false; int hitId = 0;
  for (int i=0;i<nSched;i++) {
    if (!(scheds[i].days & dayBit(t.tm_wday))) continue;
    if (mins >= scheds[i].startMin && mins < scheds[i].startMin + scheds[i].durMin) {
      shouldRun = true; hitId = scheds[i].id; break;
    }
  }

  if (now - tReport > 60000 || tReport == 0) {
    tReport = now;
    float soil = readMoisture();
    float vbat = readVolts(PIN_VBAT);
    float vsol = readVolts(PIN_VSOL);

    // safety interlocks: don't pump on a flat battery or already-wet soil
    if (vbat < 11.2 || soil > 75) shouldRun = false;

    pumpOn = shouldRun; activeSchedId = shouldRun ? hitId : 0;
    digitalWrite(PIN_PUMP, pumpOn ? HIGH : LOW);
    sendReport(soil, vbat, vsol);
  }
  delay(1000);
}
```

Three things in the webapp that will bite you

1. Manual override never reaches the device.

`public/api/device/pull-schedule.php`

It returns `{ "schedules": [...] }` and nothing else. The Override page writes to the `overrides` table, but the ESP32 has no way to read it — so the manual on/off button on the dashboard will change the DB and do absolutely nothing to the pump. Add the current override to that JSON response and have the firmware check it before the schedule loop. This is the one real functional hole in the integration.

2. The dashboard shows fake data when the device is silent.

`src/models/SensorReading.php → sample()`

With no rows in `sensor_readings` it returns 42 % moisture / 12.6 V / 68 % water level. During testing you will not be able to tell "ESP32 is reporting" from "ESP32 is dead". Add a *last seen* timestamp on the dashboard, or a stale badge when the newest reading is older than a few minutes.

3. `water_level` has no sensor in your build.

`report.php · schema.sql · dashboard`

The column and the widget exist, but nothing in your parts list measures tank level. Either add a cheap float switch (one GPIO, gives full/empty) or an HC-SR04 above the tank, or accept that the field stays `null` and hide the widget. Leaving it means the dashboard permanently shows the 68 % placeholder.

ALSO WORTH DOING

- **Never run the pump current through breadboard rails.** Contact resistance plus the motor's inrush causes voltage sag that resets the ESP32 mid-report. Solder the pump, MOSFET and the 1000 µF cap on their own strip, and take only the gate wire to the breadboard.
- **Change `DEVICE_API_KEY`** from `dev-local-device-key` in `config/device.php` before the defense — it is the only thing guarding the device endpoints.
- **XAMPP must be reachable on the LAN.** Apache has to listen on `0.0.0.0:80` and Windows Firewall needs an inbound rule for port 80. "It works in the browser but the ESP32 gets no response" is almost always the firewall.
- **Rate limit is fine.** `RateLimiter::enforce("device:$ip", 60, 60)` allows 60 requests/minute; this firmware makes about 2.
- **Bring up in stages:** buck output verified at 5.0 V → ESP32 alone on USB → moisture reading in Serial → POST reaching the dashboard → MOSFET with an LED instead of the pump → finally the real pump.